

7.1 Pod adresem <http://127.0.0.1:7001> działa prosty serwer HTTP. Serwer jest błędnie skonfigurowany, i w nagłówkach zwraca pewne wrażliwe informacje.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 7001:80 --name http1 docker.io/mazurkatarzyna/http-ex1:latest
podman run -p 7001:80 --name http1 docker.io/mazurkatarzyna/http-ex1:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 7001:80 --name http1 ghcr.io/mazurkatarzynaumcs/http-ex1:latest
podman run -p 7001:80 --name http1 ghcr.io/mazurkatarzynaumcs/http-ex1:latest
```

Po uruchomieniu serwer będzie dostępny pod adresem <http://127.0.0.1:7001>.

(b) Używając:

- Narzędzi programistycznych przeglądarki,
- Narzędzia [curl](#),
- Narzędzia [Burp Suite](#)

Podpowiedź: Sprawdź, jakie nagłówki HTTP zwróci w odpowiedzi na request typu GET serwer. Jakie informacje możesz odczytać z nagłówków?

CRTL + SHIFT + K

← → ↻ 🏠 http://127.0.0.1:7001 ☆ ⓘ 📄 ☰

🔖 Import bookmarks... 🌐 Aplikacje | M365 Copilot 📁 Parrot OS 📁 Hack The Box 📁 OSINT Services 📁 Vuln DB >>

OK

🔍 📄 Inspector 📄 Console 📄 Debugger ⬆️⬆️ Network {} Style Editor 🔄 Performance 📄 Memory 📄 Storage >> 🔴 1 📄 ☰ ✕

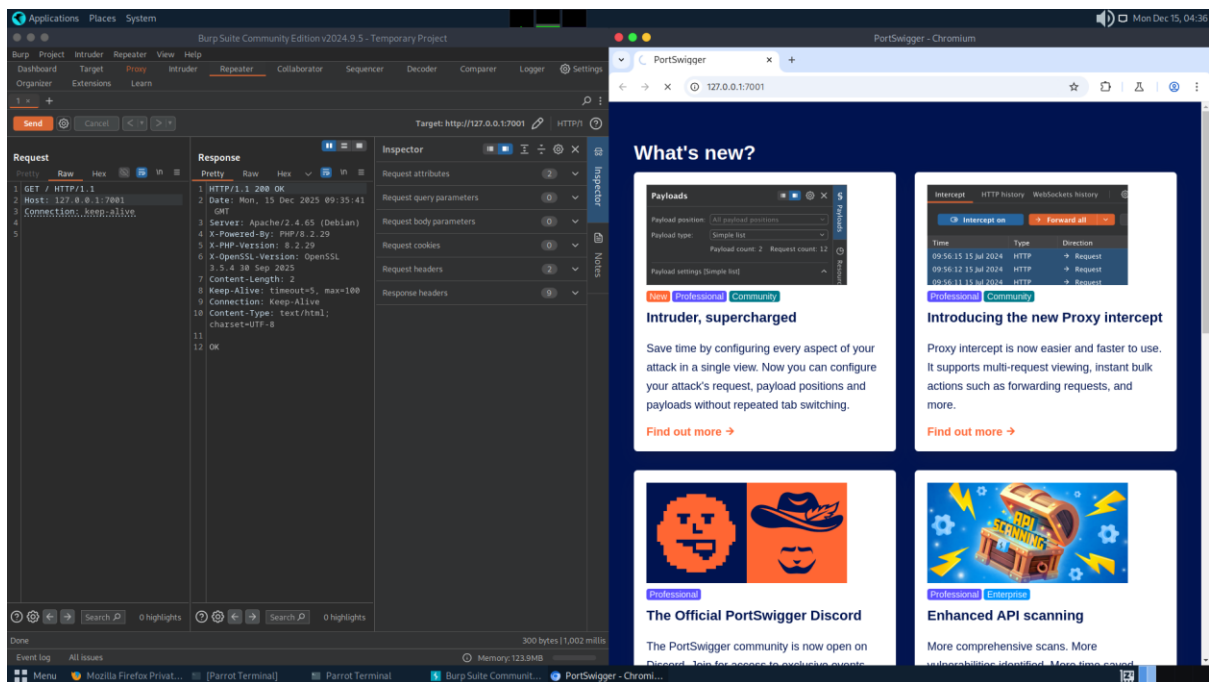
🗑️ 🔍 Filter URLs | || + 🔍 🔄 ☑️ Disable Cache No Throttling ⚙️

All HTML CSS JS XHR Fonts Images Media WS Other

Status	Method	Domain	File	Initiator	Type	Transferred	Size	0 ms	80 ms
200	GET	🔒 127.0.0.1:7001	/	document	html	300 B	2 B	1 ms	
404	GET	🔒 127.0.0.1:7001	favicon.ico	FaviconLoader.sy...	html	cached	273 B	0 ms	

🕒 2 requests | 275 B / 300 B transferred | Finish: 22 ms 📄 DOMContentLoaded: 15 ms 📄 load: 17 ms

BurpSuite



CURL

```
(.python_env) [admin@parrot]~  
└─$ curl -I http://localhost:7001  
HTTP/1.1 200 OK  
Date: Mon, 15 Dec 2025 09:37:41 GMT  
Server: Apache/2.4.65 (Debian)  
X-Powered-By: PHP/8.2.29  
X-OpenSSL-Version: OpenSSL 3.5.4 30 Sep 2025  
Content-Type: text/html; charset=UTF-8  
  
(.python_env) [admin@parrot]~  
└─$
```

7.2 Pod adresem <http://127.0.0.1:7002> działa prosty serwer HTTP, który wykrywa wykorzystywaną przeglądarkę i system operacyjny.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 7002:7002 --name http2 docker.io/mazurkatarzyna/http-ex2:latest
podman run -p 7002:7002 --name http2 docker.io/mazurkatarzyna/http-ex2:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 7002:7002 --name http2 ghcr.io/mazurkatarzynaumcs/http-ex2:latest
podman run -p 7002:7002 --name http2 ghcr.io/mazurkatarzynaumcs/http-ex2:latest
```

Po uruchomieniu serwer będzie dostępny pod adresem <http://127.0.0.1:7002>.

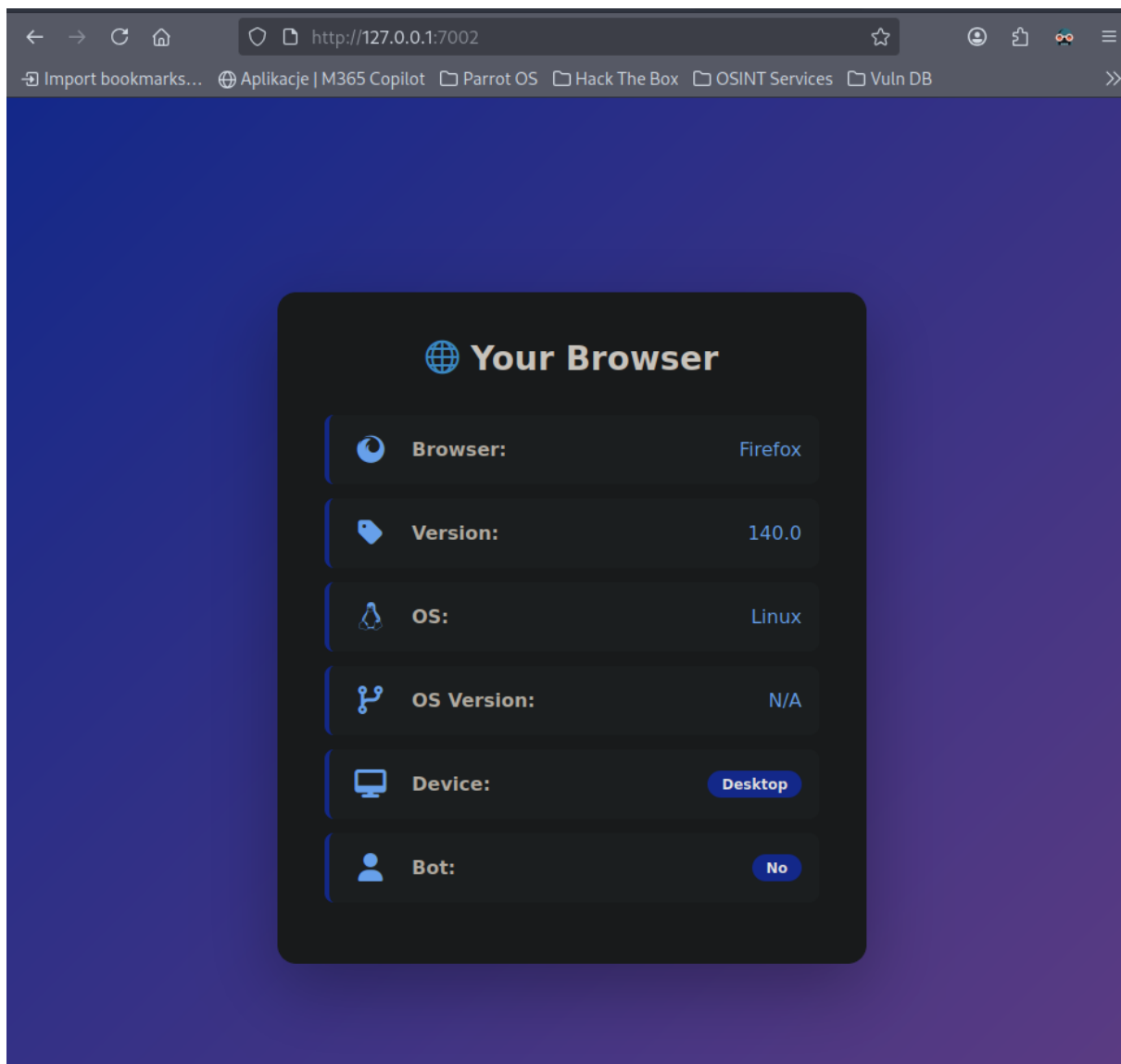
(b) Używając:

- Narzędzi programistycznych przeglądarki,
- Narzędzia [curl](#),
- Narzędzia [Burp Suite](#) (dokładniej [Burp Repeater](#))

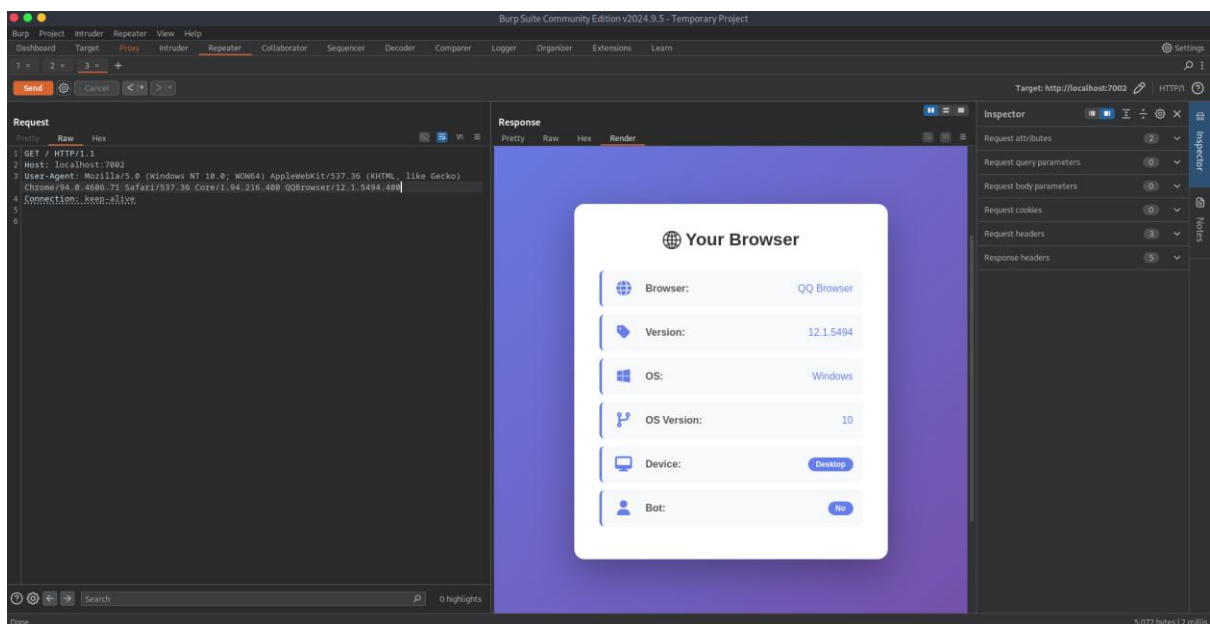
Wyślij do serwera request typu GET i podzuj się pod:

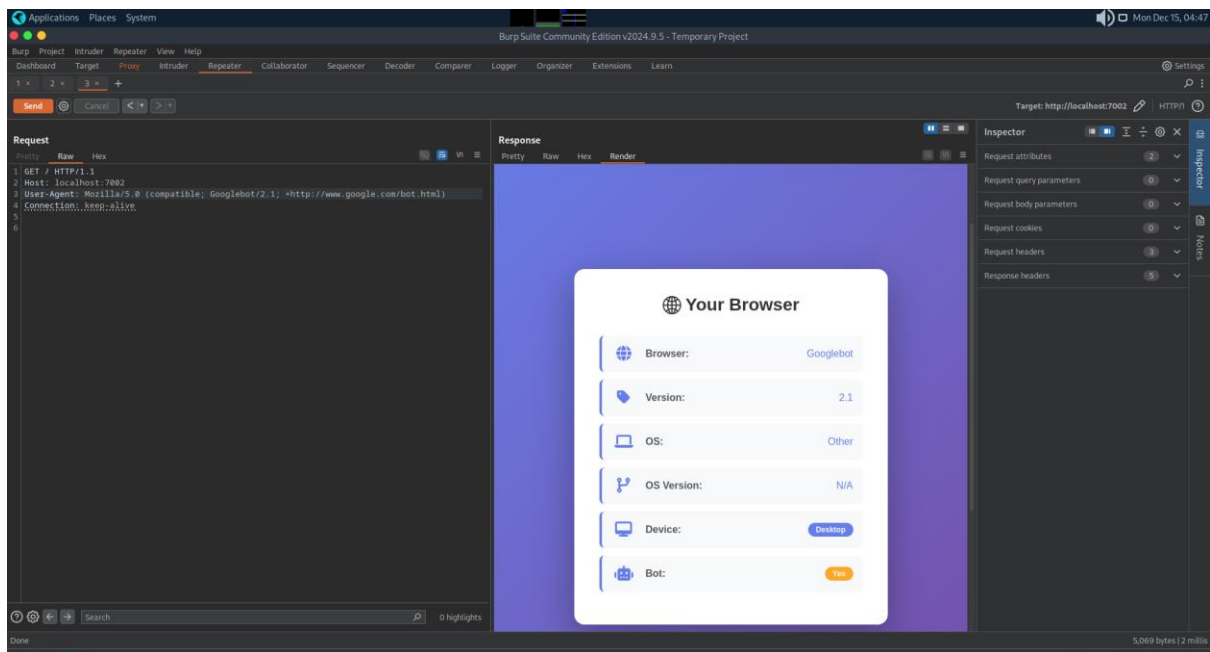
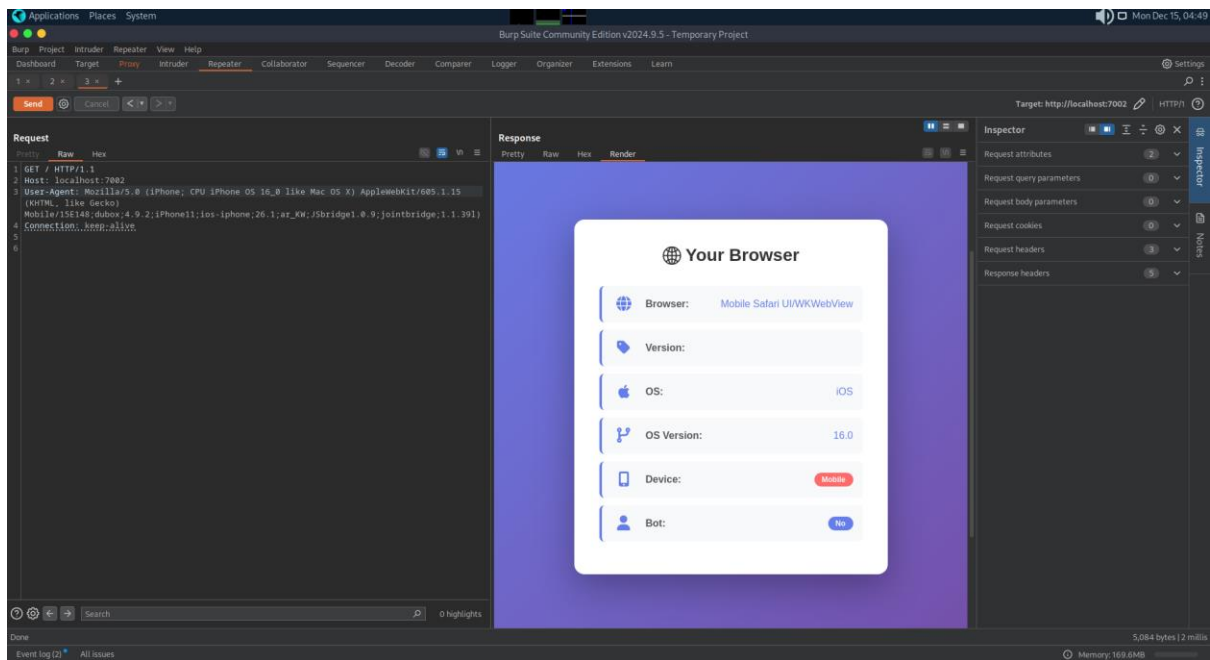
- Komputer z systemem Windows,
- iPhone 16,
- dowolnego bota

Podpowiedź: sprawdź nagłówek [User-Agent](#) oraz stronę <https://useragents.io/>.



Z BurpSuite





7.3 Pod adresem <http://127.0.0.1:7003> działa prosty serwer HTTP. Serwer posiada 100 podstron - na jednej z nich znajduje się flaga, którą musisz znaleźć.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 7003:7003 --name http3 docker.io/mazurkatarzyna/http-ex3:latest
podman run -p 7003:7003 --name http3 docker.io/mazurkatarzyna/http-ex3:latest
```

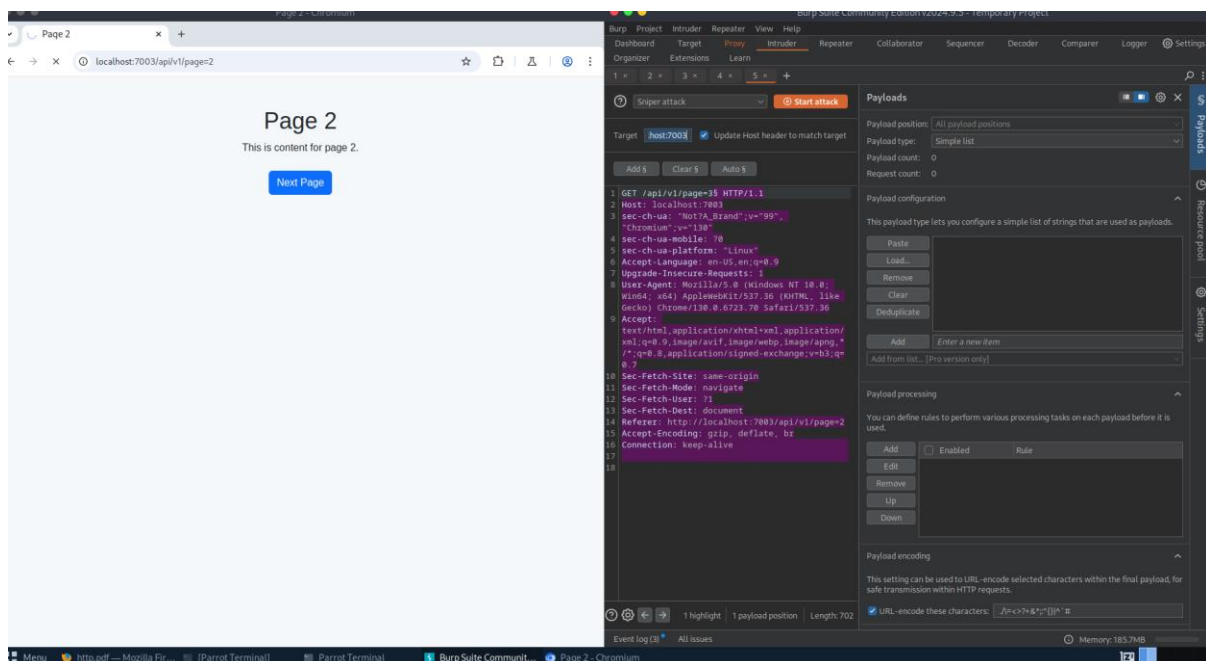
Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 7003:7003 --name http3 ghcr.io/mazurkatarzynaumcs/http-ex3:latest
podman run -p 7003:7003 --name http3 ghcr.io/mazurkatarzynaumcs/http-ex3:latest
```

Po uruchomieniu serwer będzie dostępny pod adresem <http://127.0.0.1:7003>.

(b) Używając narzędzia [Burp Intruder](#), znajdź flagę.

Podpowiedź: zwróć uwagę na kody odpowiedzi HTTP oraz wielkość odpowiedzi.



7.4 Pod adresem <http://127.0.0.1:7004> działa prosty serwer HTTP. Działająca na serwerze aplikacja zwraca informację o użytkownikach istniejących na serwerze. Twoim zadaniem jest znalezienie loginów użytkowników zarejestrowanych w aplikacji.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 7004:7004 --name http4 docker.io/mazurkatarzyna/http-ex4:latest
podman run -p 7004:7004 --name http4 docker.io/mazurkatarzyna/http-ex4:latest
```

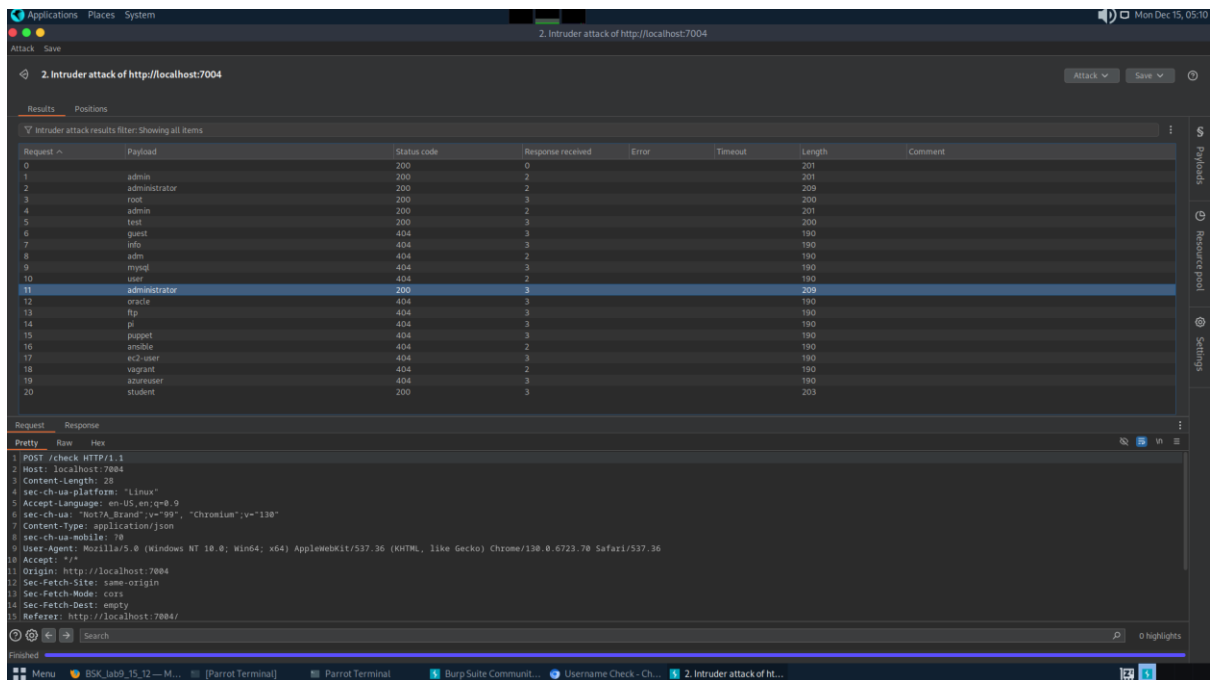
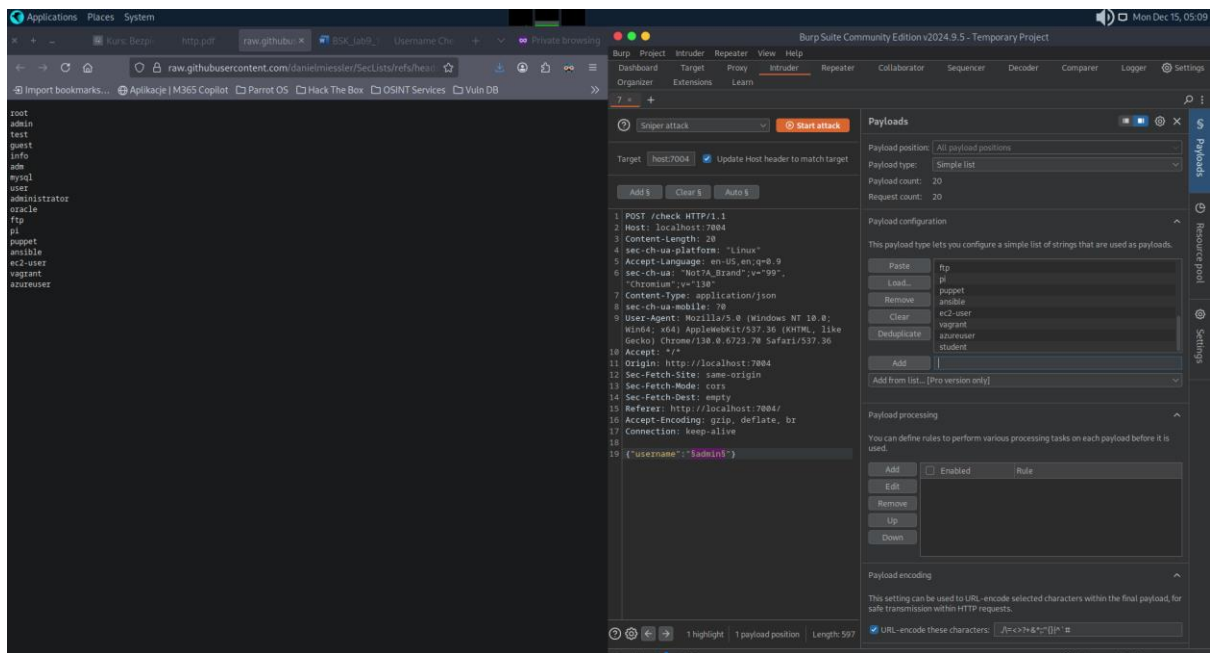
Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 7004:7004 --name http4 ghcr.io/mazurkatarzynaumcs/http-ex4:latest
podman run -p 7004:7004 --name http4 ghcr.io/mazurkatarzynaumcs/http-ex4:latest
```

Po uruchomieniu serwer będzie dostępny pod adresem <http://127.0.0.1:7004>.

(b) Używając narzędzia [Burp Intruder](#), znajdź loginy wszystkich użytkowników zarejestrowanych w aplikacji. Możesz wykorzystać słownik popularnych nazw użytkowników.

Podpowiedź: wykorzystaj atak typu [Sniper attack](#).



7.5 Pod adresem <http://127.0.0.1:7005> działa prosty serwer HTTP. Działająca na serwerze aplikacja informuje o poprawnym lub nieudanym logowaniu w zależności od podanych danych. Twoim zadaniem jest przeprowadzenie symulacji ataku bruteforce w celu odnalezienia poprawnych loginów i odpowiadających im haseł.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 7005:7005 --name http5 docker.io/mazurkatarzyna/http-ex5:latest
podman run -p 7005:7005 --name http5 docker.io/mazurkatarzyna/http-ex5:latest
```

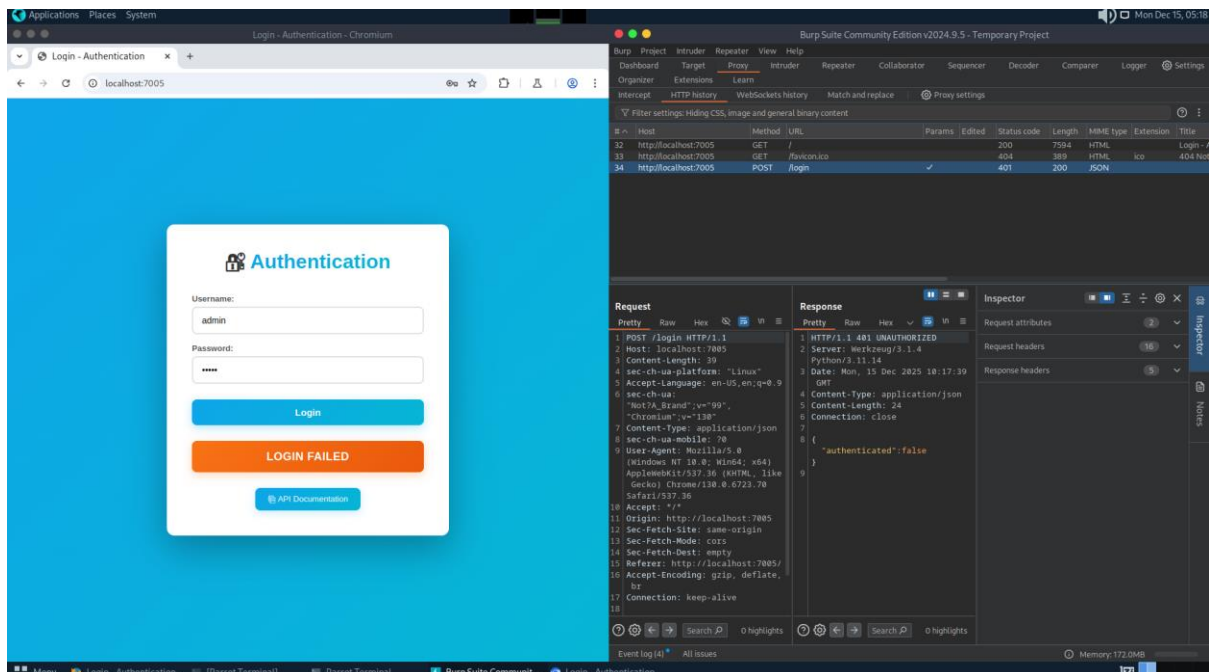
Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 7005:7005 --name http5 ghcr.io/mazurkatarzynaumcs/http-ex5:latest
podman run -p 7005:7005 --name http5 ghcr.io/mazurkatarzynaumcs/http-ex5:latest
```

Po uruchomieniu serwer będzie dostępny pod adresem <http://127.0.0.1:7005>.

(b) Używając narzędzia [Burp Intruder](#), znajdź loginy oraz hasła wszystkich użytkowników zarejestrowanych w aplikacji. Możesz wykorzystać [słownik popularnych nazw użytkowników](#) jak również [słownik popularnych haseł](#).

Podpowiedź: wykorzystaj atak typu [Pitchfork attack](#).



Burp Project Intruder Repeater View Help

Dashboard Target Proxy Intruder **Repeater** Collaborator Sequencer Decoder Comparer Logger Settings

Organizer Extensions Learn

1 x 2 x 3 x 4 x 5 x **6 x** +

Send Target: http://localhost:7005 HTTP/1

Request

```

1 POST /login HTTP/1.1
2 Host: localhost:7005
3 Content-Length: 39
4 sec-ch-ua-platform: "Linux"
5 Accept-Language: en-US,en;q=0.9
6 sec-ch-ua: "Not?A_Brand";v="99",
  "Chromium";v="130"
7 Content-Type: application/json
8 sec-ch-ua-mobile: ?0
9 User-Agent: Mozilla/5.0 (Windows NT
  10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like
  Gecko) Chrome/130.0.6723.70
  Safari/537.36
10 Accept: */*
11 Origin: http://localhost:7005
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Dest: empty
15 Referer: http://localhost:7005/
16 Accept-Encoding: gzip, deflate, br
17 Connection: keep-alive
18
19 {
  "username": "admin",
  "password": "admin"
}
          
```

Response

Inspector

- Request attributes 2
- Request query parameters 0
- Request cookies 0
- Request headers 16

? ⚙️ ⬅️ ➡️ Search 0 highlights

Ready

Event log (4) All issues Memory: 172.0MB

Authentication ...

DashboardTargetProxyIntruderRepeaterCollaboratorSequencerDecoderComparerLoggerSettings

OrganizerExtensionsLearn

7 x8 x+

?

Pitchfork attack

Start attack

Target

localhost:7005

Update Host header to match target

Add \$

Clear \$

Auto \$

1 POST /login HTTP/1.1

2 Host: localhost:7005

3 Content-Length: 39

4 sec-ch-ua-platform: "Linux"

5 Accept-Language: en-US,en;q=0.9

6 sec-ch-ua: "Not?A_Brand";v="99",

7 "Chromium";v="130"

8 Content-Type: application/json

9 sec-ch-ua-mobile: ?0

10 User-Agent: Mozilla/5.0 (Windows NT 10.0;

11 Win64; x64) AppleWebKit/537.36 (KHTML, like

12 Gecko) Chrome/130.0.6723.70 Safari/537.36

13 Accept: */*

14 Origin: http://localhost:7005

15 Sec-Fetch-Site: same-origin

16 Sec-Fetch-Mode: cors

17 Sec-Fetch-Dest: empty

18 Referer: http://localhost:7005/

19 Accept-Encoding: gzip, deflate, br

20 Connection: keep-alive

21

22 {"username":"\$admin\$","password":"\$admin\$"}

Payloads

Payload position: 1

Payload type: Simple list

Payload count: 17

Request count: 17

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste

Load...

Remove

Clear

Deduplicate

Add

Add from list... [Pro version only]

root

admin

test

guest

info

adm

mysql

user

Enter a new item

Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

Add

Edit

Remove

Up

Down

Enabled

Rule

Payload encoding

This setting can be used to URL-encode selected characters within the final payload, for safe transmission within HTTP requests.

URL-encode these characters: .\<>?+&*;'"{}|^`#

2 highlights

2 payload positions

Length: 618

7.6 Pod adresem <http://127.0.0.1:7006> działa prosty serwer HTTP. Twoim zadaniem jest znalezienie flagi, która dostępna jest jedynie dla użytkowników z uprawnieniami administratora.

(a) Uruchom serwer za pomocą poniższego polecenia:

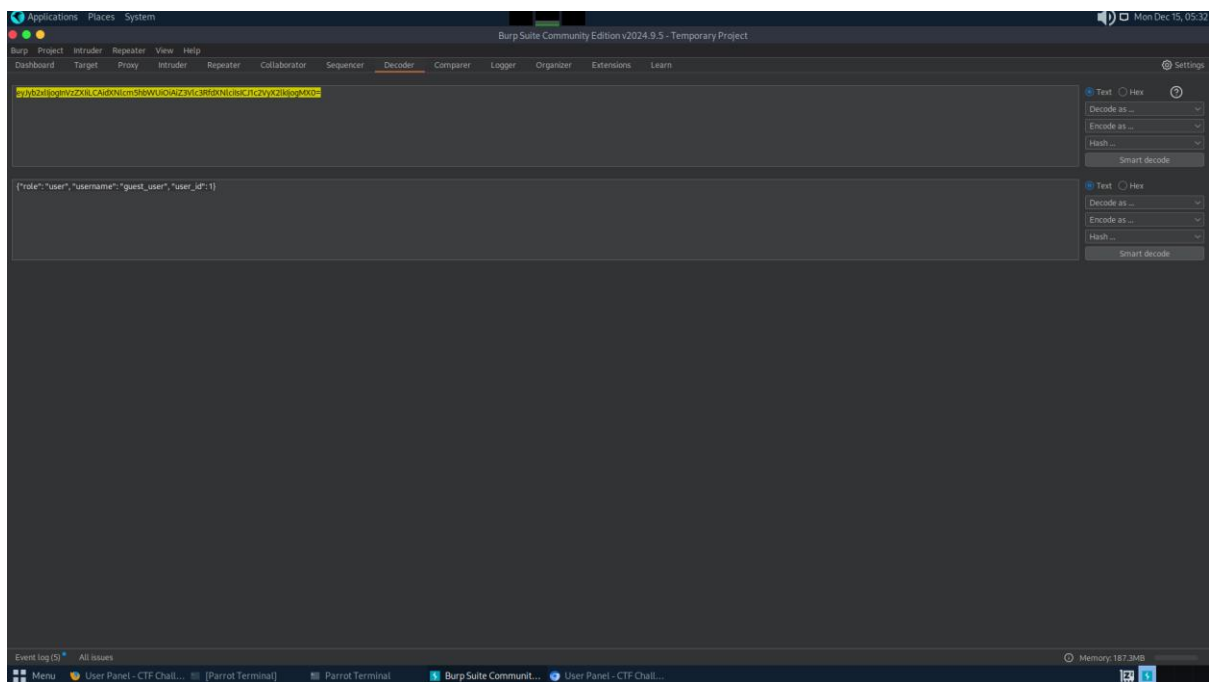
```
docker run -p 7006:7006 --name http6 docker.io/mazurkatarzyna/http-ex6:latest
podman run -p 7006:7006 --name http6 docker.io/mazurkatarzyna/http-ex6:latest
```

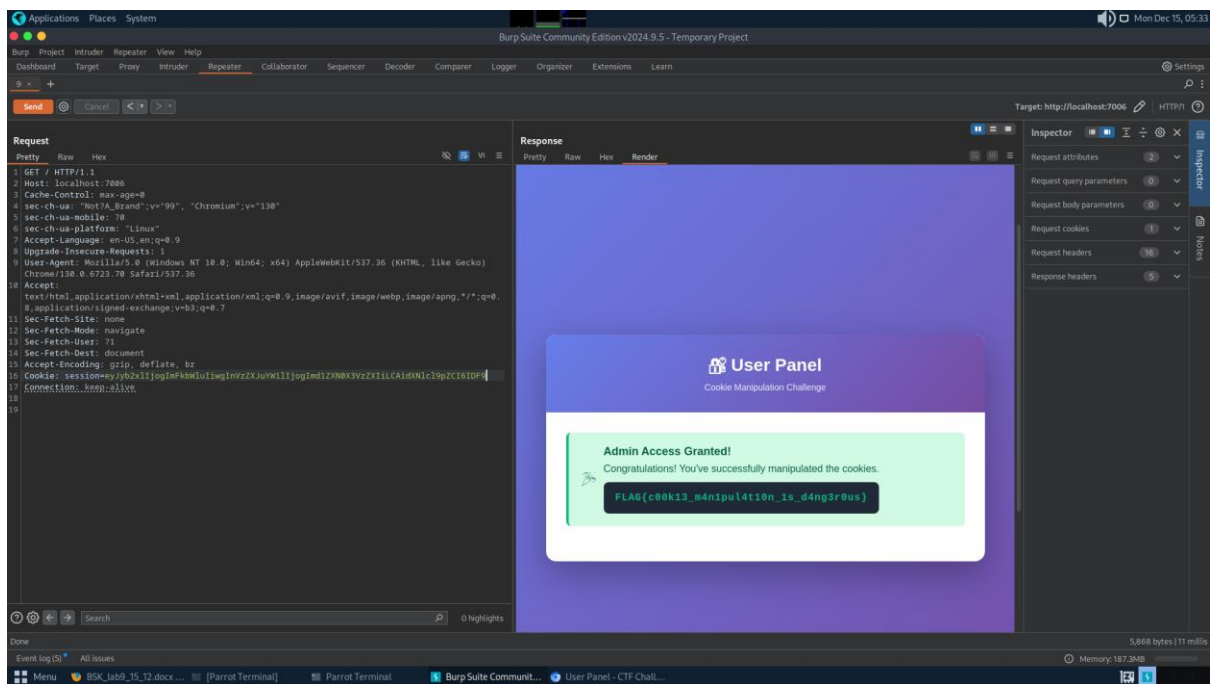
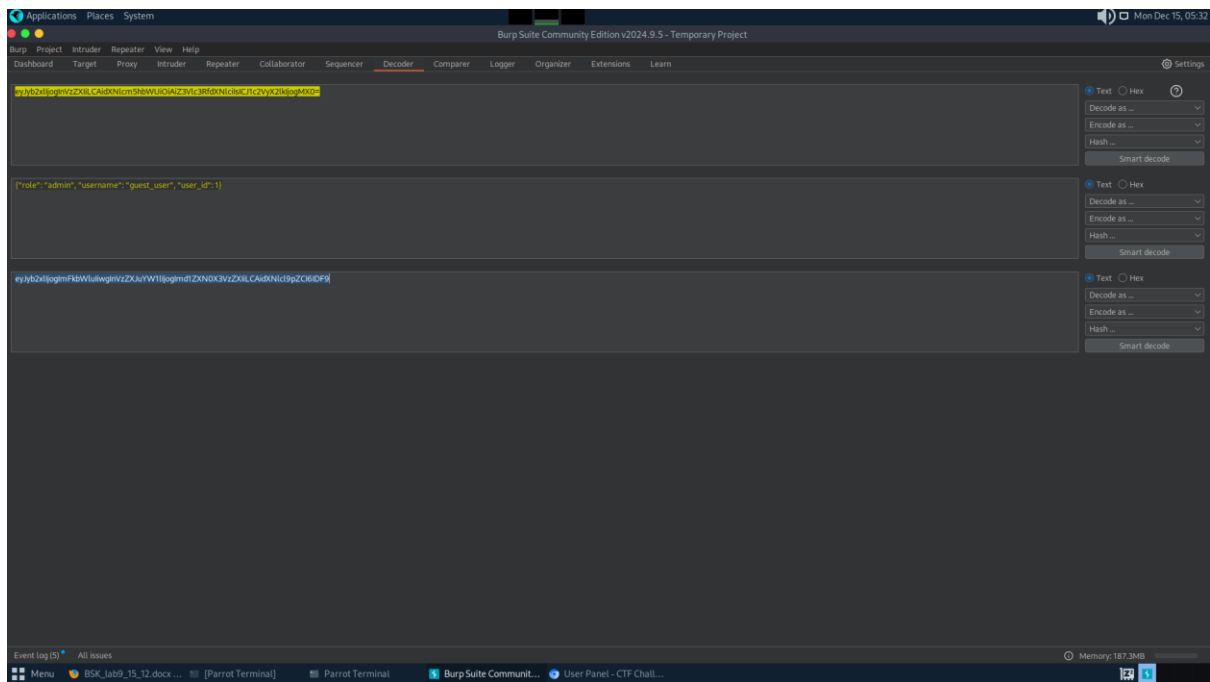
Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 7006:7006 --name http6 ghcr.io/mazurkatarzynaumcs/http-ex6:latest
podman run -p 7006:7006 --name http6 ghcr.io/mazurkatarzynaumcs/http-ex6:latest
```

Po uruchomieniu serwer będzie dostępny pod adresem <http://127.0.0.1:7006>.

- (b) Wykorzystaj narzędzie [Burp Suite](#), wyślij request typu GET do serwera, a następnie przeanalizuj nagłówki protokołu HTTP.
- (c) Sprawdź zawartość nagłówka [Cookie](#). Czy jesteś w stanie odczytać jego zawartość?
- (d) Wykorzystaj narzędzie [Burp Decoder](#) i odkoduj zawartość nagłówka [Cookie](#).
- (e) Zmodyfikuj zawartość nagłówka [Cookie](#) w taki sposób, aby uzyskać dostęp do konta administratora. Jeśli uda Ci się uzyskać dostęp do konta admina, serwer zwróci szukaną flagę.





7.7 Pod adresem `http://127.0.0.1:7007` działa prosty serwer HTTP. Serwer udostępnia dwa endpointy, w tym `http://127.0.0.1:7007/api/data`. Twoim zadaniem jest znalezienie flagi, która jest ukryta w jednej losowo wybranej metodzie protokołu HTTP.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 7007:7007 --name http7 docker.io/mazurkatarzyna/http-ex7:latest
podman run -p 7007:7007 --name http7 docker.io/mazurkatarzyna/http-ex7:latest
```

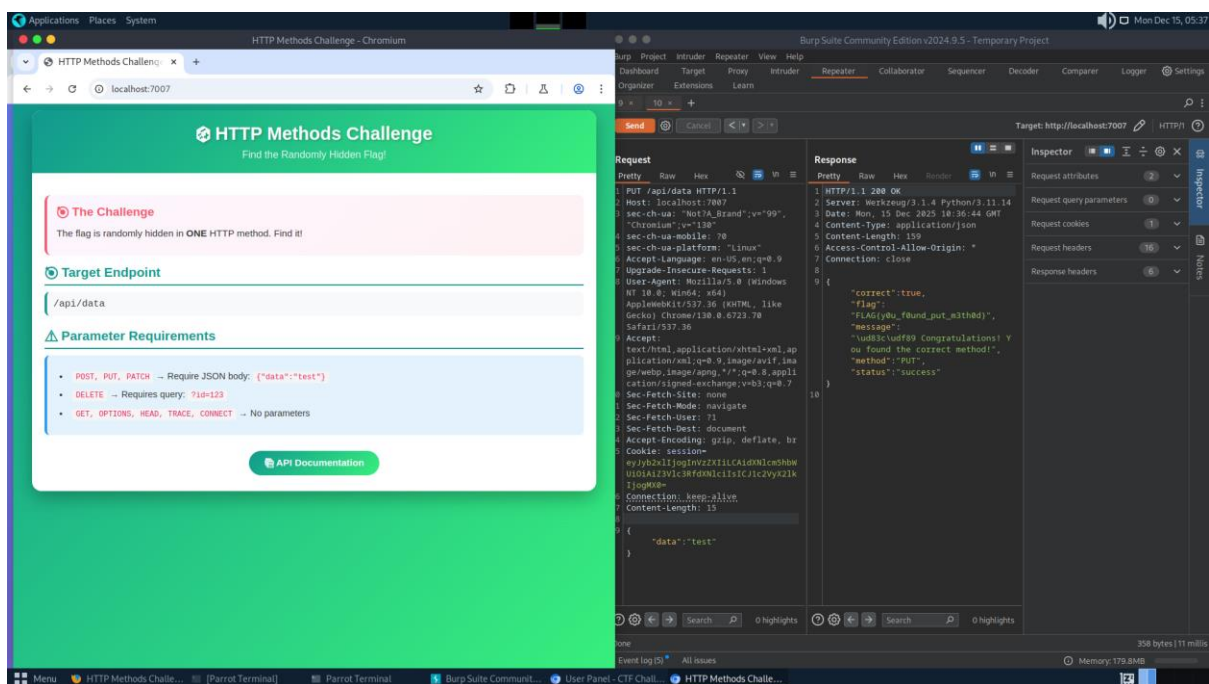
Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 7007:7007 --name http7 ghcr.io/mazurkatarzynaumcs/http-ex7:latest
podman run -p 7007:7007 --name http7 ghcr.io/mazurkatarzynaumcs/http-ex7:latest
```

Po uruchomieniu serwer będzie dostępny pod adresem `http://127.0.0.1:7007`.

(b) Wykorzystaj narzędzie [Burp Suite](#), aby wysłać do serwera request zawierający każdą dostępną metodę HTTP - w jednej z nich ukryta jest flaga.

Podpowiedź: przechwyc request z metodą GET wysłany do serwera `http://127.0.0.1:7007` w Burp Suite, a następnie użyj [Burp Repeater](#), aby odpowiednio zmodyfikować metodę w requeście HTTP (oraz sam request, jeśli dana metoda HTTP tego wymaga).



Na końcu w nagraniu info co będzie na teście